DSA ASSIGNMENT - 3B

1. AVL Trees (15 marks)

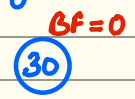
You are given an initially empty AVL tree and are given the following keys to insert in order:

30, 20, 40, 10, 25, 5

1.1 Draw the AVL tree after each insertion and show the balance factors at each node. (5 marks)
1.2 Identify the rotations performed to restore balance at each step. (5 marks)
1.3 What is the final height of the AVL tree after all insertions? (5 marks)

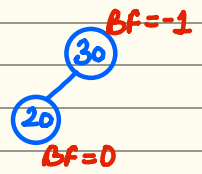
① **1.1** [Bf stands for balance factor]

(a) Adding 30



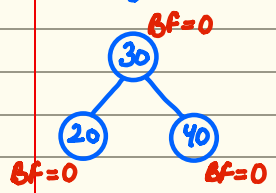
no rotation (balanced)

(b) Adding 20



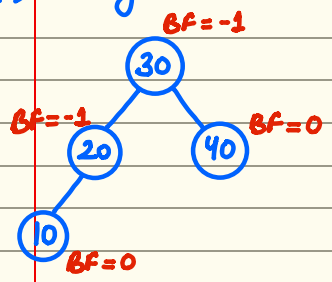
no rotation (balanced)

(c) Adding 40



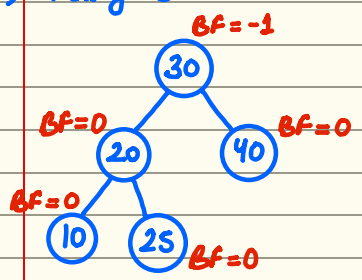
no rotation (balanced)

(d) Adding 10



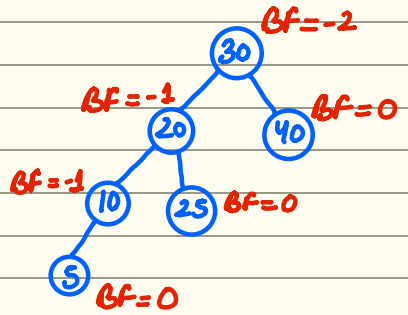
no rotation (balanced)

(e) Adding 25

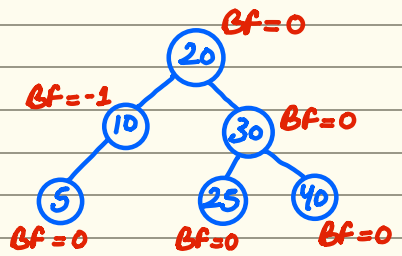


no rotation (balanced)

(f) Adding 5



Since, Bf at node 30 is -2, it is unbalanced. We need to perform single right rotation:



now, this tree is balanced.

1.2

At node 30, we performed a single right rotation.

1.3

Final height = 2

2. Graphs (15 marks)

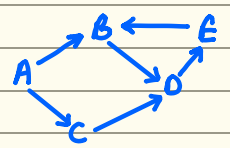
You are given the following graph G with 5 vertices A, B, C, D, E and the following directed edges:

A → B
A → C
B → D
C → D
D → E
E → B

2.1. Is this graph directed or undirected? Draw out the graph and justify your answer. (3 marks)
2.2. Represent the graph using an adjacency matrix. (3 marks)
2.3. Represent the graph using an adjacency list. (3 marks)
2.4. Compute the in-degree and out-degree for each vertex. (3 marks)
2.5. Perform Breadth-First Search (BFS) starting from vertex A. Show the step-by-step traversal and list the order in which the vertices are visited. (3 marks)

② **2.1**

Given graph:



Clearly, all the vertices have a direction edge, ∴, it is a directed graph.

2.2

	A	B	C	D	E
A	0	1	1	0	0
B	0	0	0	1	0
C	0	0	0	1	0
D	0	0	0	0	1
E	0	1	0	0	0

row : from a vertex
column : to a vertex

2.3

A → [B, C]
B → [D]
C → [D]
D → [E]
E → [B]

2.4

vertex	in-degree	out-degree
A	0	2
B	2	1
C	1	1
D	2	1
E	1	1

in-degree : no. of edges coming into the vertex.
out-degree : no. of edges going out from the vertex.

2.5

L_i : all vertices at a distance i from source vertex.

$L_0 = [A]$

$L_1 = [B, C]$

↳ neighbours of A : B, C
visited : A, B, C
add B, C

$L_2 = [D]$

↳ neighbours of B = D
visited : A, B, C, D
add D
neighbour of C = D
already visited (no need to add)

$L_3 = [E]$

↳ neighbour of D = E
visited : A, B, C, D, E
add E

∴ final order : [A, B, C, D, E]

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

3. Binary Tree Reconstruction (15 Marks)

You are given two arrays representing the inorder and preorder traversals of a binary tree. The tree contains unique values (no duplicates). Implement the program (in C/C++ without STL) to reconstruct the binary tree using the above mentioned traversals. Missing nodes are represented as "null".

Example

Input:

Preorder traversal: [3, 9, 20, 15, 7]

Inorder traversal: [9, 3, 15, 20, 7]

Output: [3, 9, 20, null, null, 15, 7]

Example

Input:

Preorder traversal: [1, 2, 3]

Inorder traversal: [2, 3, 1]

Output: Unable to construct a valid binary tree.

Explanation of code:

1. I initialized a variable invalidTree as false. It will be used further when we will check the validity of tree.
2. Then, I created the class of Tree node which has a constructor that assigns some value to the node.
3. createTreeInPre function is the main function that deals with creating a binary tree from the inorder and preorder traversal arrays. I have defined a base case as if start > end or preIndex >= total, it will return null. After that, we are validating the tree and then calling the recursive function to build the left and right subtree.
4. Then, the printKTHlevel function helps in printing the Kth level of tree which shows the null values too.
5. print function iterates for i = height of Tree times and prints all the values along with null.

Screenshot of Relevant Test Cases:

```
ode_3_2024369_ } ; if ($?) { .\code_3_2024369_ }
Enter total number of elements in the tree: 5
Enter the elements of preorder array (separated by a space): 3 9 20 15 7
Enter the elements of inorder array (separated by a space): 9 3 15 20 7
The elements of the tree are : 3 9 20 null null 15 7
PS C:\Users\HP\Downloads\C> cd "c:\Users\HP\Downloads\C\" ; if ($?) { g++ code_3_2024369_.cpp -o c
ode_3_2024369_ } ; if ($?) { .\code_3_2024369_ }
Enter total number of elements in the tree: 3
Enter the elements of preorder array (separated by a space): 1 2 3
Enter the elements of inorder array (separated by a space): 2 3 1
The elements of the tree are : 1 2 null null 3 null null
PS C:\Users\HP\Downloads\C>
```